

Práctica 03 - Espora

Buendia Elizalde Ricardo	322010215
Alvarez Reyes Janeth Alin	322279832
Oropeza Zambrano Fernando	425013902

Para nuestra práctica hicimos uso de **Java 17**, por lo que recomendamos tener esta versión instalada. En nuestro caso, trabajamos en **Linux**.

Argumentación

Para esta práctica, se hizo uso de los 3 patrones de diseño señalados en las instrucciones, que serían Iterator, Builder y Factory.

Para esta práctica vimos que era viable hacer uso del patrón Iterator en la parte de los grupos, pues en las indicaciones se plantea que vamos a tener dos grupos que se almacenan en estructuras de datos distintas, primero los Ninjas Voluntarios que se almacenan en un Array, y los Prospectos (aspirantes) que están en una HashTable. Como el programa necesita acceder a los elementos de ambos grupos para poder formar los grupos, hace perfecto el uso de este patrón. Para llegar a nuestro resultado, lo pensamos desde la perspectiva del Cliente que es la que forma los grupos, pues nos dimos cuenta de que esta clase no necesita saber si los Voluntarios están en un Array y los prospectos en una HashTable, lo único que necesita es pedir al siguiente miembro, y justo Iterator hace eso, crea una capa en la que se ocultan los detalles de la implementación de cada grupo y da una interfaz para su acceso con `next()` y `hasNext()`.

Para el patrón Builder, vimos correcto que se usará para la construcción de los paquetes de herramientas. Usamos Builder porque en la práctica necesitamos paquetes predefinidos, ya sea el básico, avanzado o el táctico, pero a su vez, también se nos es necesario poder crear un paquete personalizado, y si quisiéramos manejar todas las combinaciones en un único constructor sería ilegible y poco eficaz, por lo que el hacer uso de este patrón nos ayuda a separar el proceso de la construcción de un paquete de su representación final. Nuestra implementación fue crear un `PaqueteBuilder` que tiene los pasos generales para añadir una herramienta, junto con una clase específica para cada tipo de paquete, ya sea `PaqueteBasicoBuilder`, `PaqueteAvanzadoBuilder`, etc. donde cada una tiene las herramientas propias de la misma, además de una clase `Paquete` que hace alusión al producto final ya terminado.

Decidimos implementar el patrón Factory en la asignación del campo de entrenamiento para cada grupo, ya que esta parte depende de condiciones específicas relacionadas con la suma de habilidades de los integrantes. Es así como nos pareció la mejor forma de manejar esta lógica dentro del programa, ¿y por qué? porque una fábrica puede decidir de manera

sencilla qué campo asignar según la condición dada, evitando así que esta lógica se disperse en distintas partes del programa.

Asimismo pensamos en aplicarlo en la parte que nos habla de la formación de grupos, pues ahí también existen condiciones claras. Sin embargo, como solo se pedía una implementación y no era necesario hacerlo en ambos casos, optamos por aplicarlo en la asignación de campos, ya que nos pareció que ahí encajaba mejor y resultaba más claro el patrón.

Para compilar

Una vez clonado el repositorio y ubicados en la carpeta Practica03_espora, ejecutar:

```
$ javac -d target src/main/java/mx/unam/fciencias/espora/practica3/*.java
```

Para ejecutar

Cuando se hayan generado los `.class`, ejecutar el main:

```
$ java -cp target mx.unam.fciencias.espora.practica3.AcademiaCiencias
```